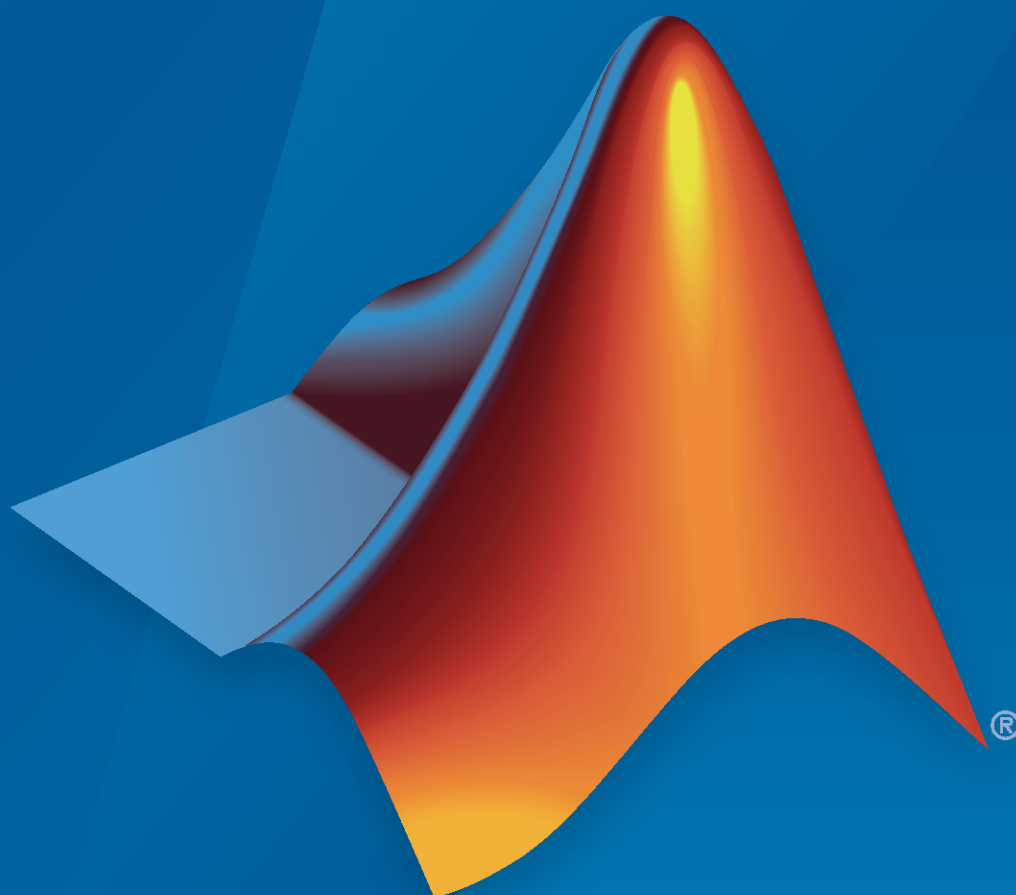


Financial Toolbox™

User's Guide



MATLAB®

R2025b



How to Contact MathWorks



Latest news: www.mathworks.com
Sales and services: www.mathworks.com/sales_and_services
User community: www.mathworks.com/matlabcentral
Technical support: www.mathworks.com/support/contact_us



Phone: 508-647-7000



The MathWorks, Inc.
1 Apple Hill Drive
Natick, MA 01760-2098

Financial Toolbox™ User's Guide

© COPYRIGHT 1995-2025 by The MathWorks, Inc.

The software described in this document is furnished under a license agreement. The software may be used or copied only under the terms of the license agreement. No part of this manual may be photocopied or reproduced in any form without prior written consent from The MathWorks, Inc.

FEDERAL ACQUISITION: This provision applies to all acquisitions of the Program and Documentation by, for, or through the federal government of the United States. By accepting delivery of the Program or Documentation, the government hereby agrees that this software or documentation qualifies as commercial computer software or commercial computer software documentation as such terms are used or defined in FAR 12.212, DFARS Part 227.72, and DFARS 252.227-7014. Accordingly, the terms and conditions of this Agreement and only those rights specified in this Agreement, shall pertain to and govern the use, modification, reproduction, release, performance, display, and disclosure of the Program and Documentation by the federal government (or other entity acquiring for or through the federal government) and shall supersede any conflicting contractual terms or conditions. If this License fails to meet the government's needs or is inconsistent in any respect with federal procurement law, the government agrees to return the Program and Documentation, unused, to The MathWorks, Inc.

Trademarks

MATLAB and Simulink are registered trademarks of The MathWorks, Inc. See www.mathworks.com/trademarks for a list of additional trademarks. Other product or brand names may be trademarks or registered trademarks of their respective holders.

Patents

MathWorks products are protected by one or more U.S. patents. Please see www.mathworks.com/patents for more information.

Revision History

October 1995	First printing	
January 1998	Second printing	Revised for Version 1.1
January 1999	Third printing	Revised for Version 2.0 (Release 11)
November 2000	Fourth printing	Revised for Version 2.1.2 (Release 12)
May 2003	Online only	Revised for Version 2.3 (Release 13)
June 2004	Online only	Revised for Version 2.4 (Release 14)
August 2004	Online only	Revised for Version 2.4.1 (Release 14+)
September 2005	Fifth printing	Revised for Version 2.5 (Release 14SP3)
March 2006	Online only	Revised for Version 3.0 (Release 2006a)
September 2006	Sixth printing	Revised for Version 3.1 (Release 2006b)
March 2007	Online only	Revised for Version 3.2 (Release 2007a)
September 2007	Online only	Revised for Version 3.3 (Release 2007b)
March 2008	Online only	Revised for Version 3.4 (Release 2008a)
October 2008	Online only	Revised for Version 3.5 (Release 2008b)
March 2009	Online only	Revised for Version 3.6 (Release 2009a)
September 2009	Online only	Revised for Version 3.7 (Release 2009b)
March 2010	Online only	Revised for Version 3.7.1 (Release 2010a)
September 2010	Online only	Revised for Version 3.8 (Release 2010b)
April 2011	Online only	Revised for Version 4.0 (Release 2011a)
September 2011	Online only	Revised for Version 4.1 (Release 2011b)
March 2012	Online only	Revised for Version 4.2 (Release 2012a)
September 2012	Online only	Revised for Version 5.0 (Release 2012b)
March 2013	Online only	Revised for Version 5.1 (Release 2013a)
September 2013	Online only	Revised for Version 5.2 (Release 2013b)
March 2014	Online only	Revised for Version 5.3 (Release 2014a)
October 2014	Online only	Revised for Version 5.4 (Release 2014b)
March 2015	Online only	Revised for Version 5.5 (Release 2015a)
September 2015	Online only	Revised for Version 5.6 (Release 2015b)
March 2016	Online only	Revised for Version 5.7 (Release 2016a)
September 2016	Online only	Revised for Version 5.8 (Release 2016b)
March 2017	Online only	Revised for Version 5.9 (Release 2017a)
September 2017	Online only	Revised for Version 5.10 (Release 2017b)
March 2018	Online only	Revised for Version 5.11 (Release 2018a)
September 2018	Online only	Revised for Version 5.12 (Release 2018b)
March 2019	Online only	Revised for Version 5.13 (Release 2019a)
September 2019	Online only	Revised for Version 5.14 (Release 2019b)
March 2020	Online only	Revised for Version 5.15 (Release 2020a)
September 2020	Online only	Revised for Version 6.0 (Release 2020b)
March 2021	Online only	Revised for Version 6.1 (Release 2021a)
September 2021	Online only	Revised for Version 6.2 (Release 2021b)
March 2022	Online only	Revised for Version 6.3 (Release 2022a)
September 2022	Online only	Revised for Version 6.4 (Release 2022b)
March 2023	Online only	Revised for Version 6.5 (Release 2023a)
September 2023	Online only	Revised for Version 23.2 (R2023b)
March 2024	Online only	Revised for Version 24.1 (R2024a)
September 2024	Online only	Revised for Version 24.2 (R2024b)
March 2025	Online only	Revised for Version 25.1 (R2025a)
September 2025	Online only	Rereleased for Version 25.2 (R2025b)

1

Getting Started

Financial Toolbox Product Description	1-2
Expected Users	1-3
Analyze Sets of Numbers Using Matrix Functions	1-4
Introduction	1-4
Key Definitions	1-4
Referencing Matrix Elements	1-4
Transposing Matrices	1-5
Matrix Algebra Refresher	1-7
Introduction	1-7
Adding and Subtracting Matrices	1-7
Multiplying Matrices	1-8
Dividing Matrices	1-11
Solving Simultaneous Linear Equations	1-11
Operating Element by Element	1-13
Using Input and Output Arguments with Functions	1-15
Input Arguments	1-15
Output Arguments	1-16

2

Performing Common Financial Tasks

Handle and Convert Dates	2-2
Date Formats	2-2
Date Conversions	2-3
Current Date and Time	2-7
Determining Specific Dates	2-8
Determining Holidays	2-8
Determining Cash-Flow Dates	2-9
Analyzing and Computing Cash Flows	2-11
Introduction	2-11
Interest Rates/Rates of Return	2-11
Present or Future Values	2-12
Depreciation	2-13
Annuities	2-13

Pricing and Computing Yields for Fixed-Income Securities	2-15
Introduction	2-15
Fixed-Income Terminology	2-15
Framework	2-18
Default Parameter Values	2-18
Coupon Date Calculations	2-20
Yield Conventions	2-21
Pricing Functions	2-21
Yield Functions	2-21
Fixed-Income Sensitivities	2-22
Treasury Bills Defined	2-25
Computing Treasury Bill Price and Yield	2-26
Introduction	2-26
Treasury Bill Repurchase Agreements	2-26
Treasury Bill Yields	2-27
Term Structure of Interest Rates	2-29
Returns with Negative Prices	2-32
Negative Price Conversion	2-32
Analysis of Negative Price Returns	2-33
Visualization of Complex Returns	2-35
Conclusion	2-38
Pricing and Analyzing Equity Derivatives	2-39
Introduction	2-39
Sensitivity Measures	2-39
Analysis Models	2-40
About Life Tables	2-44
Life Tables Theory	2-44
Case Study for Life Tables Analysis	2-46
Machine Learning for Statistical Arbitrage: Introduction	2-48
Machine Learning for Statistical Arbitrage I: Data Management and Visualization	2-50
Machine Learning for Statistical Arbitrage II: Feature Engineering and Model Development	2-59
Machine Learning for Statistical Arbitrage III: Training, Tuning, and Prediction	2-69
Backtest Deep Learning Model for Algorithmic Trading of Limit Order Book Data	2-78

Analyzing Portfolios	3-2
Portfolio Optimization Functions	3-3
Portfolio Construction Examples	3-5
Introduction	3-5
Efficient Frontier Example	3-5
Portfolio Selection and Risk Aversion	3-7
Introduction	3-7
Optimal Risky Portfolio	3-8
portopt Migration to Portfolio Object	3-11
Migrate portopt Without Output Arguments	3-11
Migrate portopt with Output Arguments	3-12
Migrate portopt for Target Returns Within Range of Efficient Portfolio Returns	3-13
Migrate portopt for Target Return Outside Range of Efficient Portfolio Returns	3-14
Migrate portopt Using portcons Output for ConSet	3-15
Integrate Output from portcons, pcalims, pcglims, and pcgcomp with a Portfolio Object	3-17
Constraint Specification Using a Portfolio Object	3-19
Constraints for Efficient Frontier	3-19
Linear Constraint Equations	3-21
Specifying Group Constraints	3-24
Active Returns and Tracking Error Efficient Frontier	3-27

Mean-Variance Portfolio Optimization Tools

Portfolio Optimization Theory	4-4
Portfolio Optimization Problems	4-4
Portfolio Problem Specification	4-4
Return Proxy	4-5
Risk Proxy	4-6
Supported Constraints for Portfolio Optimization Using Portfolio Objects	4-9
Linear Inequality Constraints	4-9
Linear Equality Constraints	4-10
'Simple' Bound Constraints	4-10
'Conditional' Bound Constraints	4-11
Budget Constraints	4-11
Conditional Budget Constraints	4-12
Group Constraints	4-12

Group Ratio Constraints	4-13
Average Turnover Constraints	4-14
One-Way Turnover Constraints	4-14
Tracking Error Constraints	4-15
Cardinality Constraints	4-16
Default Portfolio Problem	4-17
Portfolio Object Workflow	4-18
Portfolio Object	4-20
Portfolio Object Properties and Functions	4-20
Working with Portfolio Objects	4-20
Setting and Getting Properties	4-20
Displaying Portfolio Objects	4-21
Saving and Loading Portfolio Objects	4-21
Estimating Efficient Portfolios and Frontiers	4-21
Arrays of Portfolio Objects	4-22
Subclassing Portfolio Objects	4-23
Conventions for Representation of Data	4-23
Creating the Portfolio Object	4-25
Syntax	4-25
Portfolio Problem Sufficiency	4-25
Portfolio Function Examples	4-26
Common Operations on the Portfolio Object	4-33
Naming a Portfolio Object	4-33
Configuring the Assets in the Asset Universe	4-33
Setting Up a List of Asset Identifiers	4-33
Truncating and Padding Asset Lists	4-35
Setting Up an Initial or Current Portfolio	4-37
Setting Up a Tracking Portfolio	4-40
Asset Returns and Moments of Asset Returns Using Portfolio Object	4-42
Assignment Using the Portfolio Function	4-42
Assignment Using the setAssetMoments Function	4-43
Scalar Expansion of Arguments	4-44
Estimating Asset Moments from Prices or Returns	4-45
Estimating Asset Moments with Missing Data	4-47
Estimating Asset Moments from Time Series Data	4-49
Working with a Riskless Asset	4-52
Working with Transaction Costs	4-54
Setting Transaction Costs Using the Portfolio Function	4-54
Setting Transaction Costs Using the setCosts Function	4-54
Setting Transaction Costs with Scalar Expansion	4-56
Working with Portfolio Constraints Using Defaults	4-58
Setting Default Constraints for Portfolio Weights Using Portfolio Object	4-58

Working with 'Simple' Bound Constraints Using Portfolio Object	4-62
Setting 'Simple' Bounds Using the Portfolio Function	4-62
Setting 'Simple' Bounds Using the setBounds Function	4-62
Setting 'Simple' Bounds Using the Portfolio Function or setBounds Function	4-63
Working with Budget Constraints Using Portfolio Object	4-65
Setting Budget Constraints Using the Portfolio Function	4-65
Setting Budget Constraints Using the setBudget Function	4-65
Working with Conditional Budget Constraints Using Portfolio Object . .	4-67
Setting Conditional Budget Constraints Using the Portfolio Function . . .	4-67
Setting Conditional Budget Constraints Using the setConditionalBudget Function	4-67
Working with Group Constraints Using Portfolio Object	4-69
Setting Group Constraints Using the Portfolio Function	4-69
Setting Group Constraints Using the setGroups and addGroups Functions	4-69
Working with Group Ratio Constraints Using Portfolio Object	4-72
Setting Group Ratio Constraints Using the Portfolio Function	4-72
Setting Group Ratio Constraints Using the setGroupRatio and addGroupRatio Functions	4-73
Working with Linear Equality Constraints Using Portfolio Object	4-75
Setting Linear Equality Constraints Using the Portfolio Function	4-75
Setting Linear Equality Constraints Using the setEquality and addEquality Functions	4-75
Working with Linear Inequality Constraints Using Portfolio Object	4-78
Setting Linear Inequality Constraints Using the Portfolio Function	4-78
Setting Linear Inequality Constraints Using the setInequality and addInequality Functions	4-78
Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using Portfolio Objects	4-81
Setting 'Conditional' BoundType Constraints Using the setBounds Function	4-81
Setting the Limits on the Number of Assets Invested Using the setMinMaxNumAssets Function	4-82
Working with Average Turnover Constraints Using Portfolio Object . . .	4-85
Setting Average Turnover Constraints Using the Portfolio Function	4-85
Setting Average Turnover Constraints Using the setTurnover Function . .	4-85
Working with One-Way Turnover Constraints Using Portfolio Object . . .	4-88
Setting One-Way Turnover Constraints Using the Portfolio Function	4-88
Setting Turnover Constraints Using the setOneWayTurnover Function . .	4-88
Working with Tracking Error Constraints Using Portfolio Object	4-91
Setting Tracking Error Constraints Using the Portfolio Function	4-91
Setting Tracking Error Constraints Using the setTrackingError Function	4-91

Validate the Portfolio Problem for Portfolio Object	4-94
Validating a Portfolio Set	4-94
Validating Portfolios	4-95
Estimate Efficient Portfolios for Entire Efficient Frontier for Portfolio Object	4-98
Obtaining Portfolios Along the Entire Efficient Frontier	4-99
Obtaining Endpoints of the Efficient Frontier	4-102
Obtaining Efficient Portfolios for Target Returns	4-105
Obtaining Efficient Portfolios for Target Risks	4-108
Efficient Portfolio That Maximizes Sharpe Ratio	4-111
Choosing and Controlling the Solver for Mean-Variance Portfolio Optimization	4-114
Using 'lcprog' and 'quadprog'	4-114
Using the Mixed Integer Nonlinear Programming (MINLP) Solver	4-115
Solver Guidelines for Portfolio Objects	4-115
Solver Guidelines for Custom Objective Problems Using Portfolio Objects	4-119
Estimate Efficient Frontiers for Portfolio Object	4-122
Obtaining Portfolio Risks and Returns	4-122
Plotting the Efficient Frontier for a Portfolio Object	4-125
Postprocessing Results to Set Up Tradable Portfolios	4-130
When to Use Portfolio Objects Over Optimization Toolbox	4-132
Always Use Portfolio, PortfolioCVaR, or PortfolioMAD Object	4-134
Preferred Use of Portfolio, PortfolioCVaR, or PortfolioMAD Object	4-135
Use Optimization Toolbox	4-136
Comparison of Methods for Covariance Estimation	4-138
Choose MINLP Solvers for Portfolio Problems	4-140
Troubleshooting Portfolio Optimization Results	4-145
Portfolio Object Destroyed When Modifying	4-145
Optimization Fails with "Bad Pivot" Message	4-145
Speed of Optimization	4-145
Matrix Incompatibility and "Non-Conformable" Errors	4-145
Missing Data Estimation Fails	4-145
mv_optim_transform Errors	4-145
solveContinuousCustomObjProb or solveMICustomObjProb Errors	4-146
Efficient Portfolios Do Not Make Sense	4-146
Efficient Frontiers Do Not Make Sense	4-146
Troubleshooting estimateCustomObjectivePortfolio	4-148
Troubleshooting for Setting 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints	4-148

Role of Convexity in Portfolio Problems	4-157
Examples of Convex Functions	4-158
Examples of Concave Functions	4-159
Examples of Nonconvex Functions	4-159
Portfolio Optimization Examples Using Financial Toolbox	4-161
Asset Allocation Case Study	4-180
Portfolio Optimization with Semicontinuous and Cardinality Constraints	4-190
Portfolio Optimization Against a Benchmark	4-202
Portfolio Analysis with Turnover Constraints	4-211
Leverage in Portfolio Optimization with a Risk-Free Asset	4-217
Black-Litterman Portfolio Optimization Using Financial Toolbox	4-222
Portfolio Optimization Using Factor Models	4-231
Backtest Investment Strategies Using Financial Toolbox	4-238
Backtest Investment Strategies with Trading Signals	4-251
Portfolio Optimization Using Social Performance Measure	4-264
Diversify ESG Portfolios	4-271
Risk Budgeting Portfolio	4-286
Backtest Using Risk-Based Equity Indexation	4-291
Create Hierarchical Risk Parity Portfolio	4-296
Backtest Strategies Using Deep Learning	4-302
Backtest with Brinson Attribution to Evaluate Portfolio Performance	4-315
Analyze Performance Attribution Using Brinson Model	4-323
Diversify Portfolios Using Custom Objective	4-331
Solve Tracking Error Portfolio Problems	4-343
Solve Problem for Minimum Tracking Error with Net Return Constraint	4-349
Solve Robust Portfolio Maximum Return Problem with Ellipsoidal Uncertainty	4-351
Risk Parity or Budgeting with Constraints	4-357

Single Period Goal-Based Wealth Management	4-362
Dynamic Portfolio Allocation in Goal-Based Wealth Management for Multiple Time Periods	4-367
Multiperiod Goal-Based Wealth Management Using Reinforcement Learning	4-379
Compare Performance of Covariance Denoising with Factor Modeling Using Backtesting	4-394
Mixed-Integer Mean-Variance Portfolio Optimization Problem	4-402
Deep Reinforcement Learning for Optimal Trade Execution	4-407
Backtest Investment Strategies Using datetime and calendarDuration	4-451
Adding Constraints to Satisfy UCITS Directive	4-457

CVaR Portfolio Optimization Tools

5

Portfolio Optimization Theory	5-3
Portfolio Optimization Problems	5-3
Portfolio Problem Specification	5-3
Return Proxy	5-4
Risk Proxy	5-5
Supported Constraints for Portfolio Optimization Using PortfolioCVaR	
Object	5-8
Linear Inequality Constraints	5-8
Linear Equality Constraints	5-9
'Simple' Bound Constraints	5-9
'Conditional' Bound Constraints	5-10
Budget Constraints	5-10
Conditional Budget Constraints	5-11
Group Constraints	5-11
Group Ratio Constraints	5-12
Average Turnover Constraints	5-13
One-way Turnover Constraints	5-13
Cardinality Constraints	5-14
Default Portfolio Problem	5-15
PortfolioCVaR Object Workflow	5-16
PortfolioCVaR Object	5-17
PortfolioCVaR Object Properties and Functions	5-17
Working with PortfolioCVaR Objects	5-17
Setting and Getting Properties	5-18
Displaying PortfolioCVaR Objects	5-18

Saving and Loading PortfolioCVaR Objects	5-18
Estimating Efficient Portfolios and Frontiers	5-18
Arrays of PortfolioCVaR Objects	5-19
Subclassing PortfolioCVaR Objects	5-20
Conventions for Representation of Data	5-20
Creating the PortfolioCVaR Object	5-22
Syntax	5-22
PortfolioCVaR Problem Sufficiency	5-22
PortfolioCVaR Function Examples	5-23
Common Operations on the PortfolioCVaR Object	5-29
Naming a PortfolioCVaR Object	5-29
Configuring the Assets in the Asset Universe	5-29
Setting Up a List of Asset Identifiers	5-29
Truncating and Padding Asset Lists	5-31
Setting Up an Initial or Current Portfolio	5-33
Asset Returns and Scenarios Using PortfolioCVaR Object	5-36
How Stochastic Optimization Works	5-36
What Are Scenarios?	5-36
Setting Scenarios Using the PortfolioCVaR Function	5-37
Setting Scenarios Using the setScenarios Function	5-38
Estimating the Mean and Covariance of Scenarios	5-38
Simulating Normal Scenarios	5-39
Simulating Normal Scenarios from Returns or Prices	5-39
Simulating Normal Scenarios with Missing Data	5-40
Simulating Normal Scenarios from Time Series Data	5-42
Simulating Normal Scenarios with Mean and Covariance	5-44
Working with a Riskless Asset	5-46
Working with Transaction Costs	5-47
Setting Transaction Costs Using the PortfolioCVaR Function	5-47
Setting Transaction Costs Using the setCosts Function	5-47
Setting Transaction Costs with Scalar Expansion	5-49
Working with CVaR Portfolio Constraints Using Defaults	5-51
Setting Default Constraints for Portfolio Weights Using PortfolioCVaR Object	5-51
Working with 'Simple' Bound Constraints Using PortfolioCVaR Object	5-55
Setting 'Simple' Bounds Using the PortfolioCVaR Function	5-55
Setting 'Simple' Bounds Using the setBounds Function	5-55
Setting 'Simple' Bounds Using the PortfolioCVaR Function or setBounds Function	5-56
Working with Budget Constraints Using PortfolioCVaR Object	5-58
Setting Budget Constraints Using the PortfolioCVaR Function	5-58
Setting Budget Constraints Using the setBudget Function	5-58

Working with Conditional Budget Constraints Using PortfolioCvAR Object	
.....	5-60
Setting Conditional Budget Constraints Using the PortfolioCvAR Function	
.....	5-60
Setting Conditional Budget Constraints Using the setConditionalBudget	
Function	5-60
Working with Group Constraints Using PortfolioCvAR Object	5-62
Setting Group Constraints Using the PortfolioCvAR Function	5-62
Setting Group Constraints Using the setGroups and addGroups Functions	
.....	5-62
Working with Group Ratio Constraints Using PortfolioCvAR Object	5-65
Setting Group Ratio Constraints Using the PortfolioCvAR Function	5-65
Setting Group Ratio Constraints Using the setGroupRatio and	
addGroupRatio Functions	5-66
Working with Linear Equality Constraints Using PortfolioCvAR Object	
.....	5-68
Setting Linear Equality Constraints Using the PortfolioCvAR Function ..	5-68
Setting Linear Equality Constraints Using the setEquality and addEquality	
Functions	5-68
Working with Linear Inequality Constraints Using PortfolioCvAR Object	
.....	5-70
Setting Linear Inequality Constraints Using the PortfolioCvAR Function	
.....	5-70
Setting Linear Inequality Constraints Using the setInequality and	
addInequality Functions	5-70
Working with 'Conditional' BoundType, MinNumAssets, and	
 MaxNumAssets Constraints Using PortfolioCvAR Objects	5-72
Setting 'Conditional' BoundType Constraints Using the setBounds Function	
.....	5-72
Setting the Limits on the Number of Assets Invested Using the	
setMinMaxNumAssets Function	5-73
Working with Average Turnover Constraints Using PortfolioCvAR Object	
.....	5-75
Setting Average Turnover Constraints Using the PortfolioCvAR Function	
.....	5-75
Setting Average Turnover Constraints Using the setTurnover Function ..	5-75
Working with One-Way Turnover Constraints Using PortfolioCvAR Object	
.....	5-78
Setting One-Way Turnover Constraints Using the PortfolioCvAR Function	
.....	5-78
Setting Turnover Constraints Using the setOneWayTurnover Function ..	5-78
Validate the CVaR Portfolio Problem	5-81
Validating a CVaR Portfolio Set	5-81
Validating CVaR Portfolios	5-82
Estimate Efficient Portfolios for Entire Frontier for PortfolioCvAR Object	
.....	5-85

Obtaining Portfolios Along the Entire Efficient Frontier	5-86
Obtaining Endpoints of the Efficient Frontier	5-89
Obtaining Efficient Portfolios for Target Returns	5-92
Obtaining Efficient Portfolios for Target Risks	5-95
Choosing and Controlling the Solver for PortfolioCVaR Optimizations .	5-98
Using 'TrustRegionCP', 'ExtendedCP', and 'cuttingplane' SolverTypes . . .	5-98
Using 'fmincon' SolverType	5-99
Using the Mixed Integer Nonlinear Programming (MINLP) Solver	5-100
Solver Guidelines for PortfolioCVaR Objects	5-100
Estimate Efficient Frontiers for PortfolioCVaR Object	5-105
Obtaining CVaR Portfolio Risks and Returns	5-105
Obtaining Portfolio Standard Deviation and VaR	5-106
Plotting the Efficient Frontier for a PortfolioCVaR Object	5-109
Postprocessing Results to Set Up Tradable Portfolios	5-115
Working with Other Portfolio Objects	5-118
Troubleshooting CVaR Portfolio Optimization Results	5-121
PortfolioCVaR Object Destroyed When Modifying	5-121
Matrix Incompatibility and "Non-Conformable" Errors	5-121
CVaR Portfolio Optimization Warns About "Max Iterations"	5-121
CVaR Portfolio Optimization Errors with "Could Not Solve" Message . .	5-122
Missing Data Estimation Fails	5-122
cvar_optim_transform Errors	5-122
Efficient Portfolios Do Not Make Sense	5-123
Hedging Using CVaR Portfolio Optimization	5-125
Compute Maximum Reward-to-Risk Ratio for CVaR Portfolio	5-137
Mixed-Integer CVaR Portfolio Optimization Problem	5-141
Bond Portfolio CVaR Optimization Using Diebold-Li Model	5-146

MAD Portfolio Optimization Tools

6

Portfolio Optimization Theory	6-3
Portfolio Optimization Problems	6-3
Portfolio Problem Specification	6-3
Return Proxy	6-4
Risk Proxy	6-5

Supported Constraints for Portfolio Optimization Using PortfolioMAD	
Object	6-8
Linear Inequality Constraints	6-8
Linear Equality Constraints	6-9
'Simple' Bound Constraints	6-9
'Conditional' Bound Constraints	6-10
Budget Constraints	6-10
Conditional Budget Constraints	6-11
Group Constraints	6-11
Group Ratio Constraints	6-12
Average Turnover Constraints	6-13
One-way Turnover Constraints	6-13
Cardinality Constraints	6-14
Default Portfolio Problem	6-15
PortfolioMAD Object Workflow	6-16
PortfolioMAD Object	6-17
PortfolioMAD Object Properties and Functions	6-17
Working with PortfolioMAD Objects	6-17
Setting and Getting Properties	6-18
Displaying PortfolioMAD Objects	6-18
Saving and Loading PortfolioMAD Objects	6-18
Estimating Efficient Portfolios and Frontiers	6-18
Arrays of PortfolioMAD Objects	6-19
Subclassing PortfolioMAD Objects	6-20
Conventions for Representation of Data	6-20
Creating the PortfolioMAD Object	6-22
Syntax	6-22
PortfolioMAD Problem Sufficiency	6-22
PortfolioMAD Function Examples	6-23
Common Operations on the PortfolioMAD Object	6-29
Naming a PortfolioMAD Object	6-29
Configuring the Assets in the Asset Universe	6-29
Setting Up a List of Asset Identifiers	6-29
Truncating and Padding Asset Lists	6-31
Setting Up an Initial or Current Portfolio	6-33
Asset Returns and Scenarios Using PortfolioMAD Object	6-35
How Stochastic Optimization Works	6-35
What Are Scenarios?	6-35
Setting Scenarios Using the PortfolioMAD Function	6-36
Setting Scenarios Using the setScenarios Function	6-37
Estimating the Mean and Covariance of Scenarios	6-37
Simulating Normal Scenarios	6-38
Simulating Normal Scenarios from Returns or Prices	6-38
Simulating Normal Scenarios with Missing Data	6-39
Simulating Normal Scenarios from Time Series Data	6-41
Simulating Normal Scenarios with Mean and Covariance	6-43
Working with a Riskless Asset	6-45

Working with Transaction Costs	6-46
Setting Transaction Costs Using the PortfolioMAD Function	6-46
Setting Transaction Costs Using the setCosts Function	6-46
Setting Transaction Costs with Scalar Expansion	6-48
Working with MAD Portfolio Constraints Using Defaults	6-50
Setting Default Constraints for Portfolio Weights Using PortfolioMAD Object	6-50
Working with 'Simple' Bound Constraints Using PortfolioMAD Object .	6-54
Setting 'Simple' Bounds Using the PortfolioMAD Function	6-54
Setting 'Simple' Bounds Using the setBounds Function	6-54
Setting 'Simple' Bounds Using the PortfolioMAD Function or setBounds Function	6-55
Working with Budget Constraints Using PortfolioMAD Object	6-57
Setting Budget Constraints Using the PortfolioMAD Function	6-57
Setting Budget Constraints Using the setBudget Function	6-57
Working with Conditional Budget Constraints Using PortfolioMAD Object	6-59
Setting Conditional Budget Constraints Using the PortfolioMAD Function	6-59
Setting Conditional Budget Constraints Using the setConditionalBudget Function	6-59
Working with Group Constraints Using PortfolioMAD Object	6-61
Setting Group Constraints Using the PortfolioMAD Function	6-61
Setting Group Constraints Using the setGroups and addGroups Functions	6-61
Working with Group Ratio Constraints Using PortfolioMAD Object	6-64
Setting Group Ratio Constraints Using the PortfolioMAD Function	6-64
Setting Group Ratio Constraints Using the setGroupRatio and addGroupRatio Functions	6-65
Working with Linear Equality Constraints Using PortfolioMAD Object	6-67
Setting Linear Equality Constraints Using the PortfolioMAD Function ..	6-67
Setting Linear Equality Constraints Using the setEquality and addEquality Functions	6-67
Working with Linear Inequality Constraints Using PortfolioMAD Object	6-69
Setting Linear Inequality Constraints Using the PortfolioMAD Function .	6-69
Setting Linear Inequality Constraints Using the setInequality and addInequality Functions	6-69
Working with 'Conditional' BoundType, MinNumAssets, and MaxNumAssets Constraints Using PortfolioMAD Objects	6-71
Setting 'Conditional' BoundType Constraints Using the setBounds Function	6-71
Setting the Limits on the Number of Assets Invested Using the setMinMaxNumAssets Function	6-72

Working with Average Turnover Constraints Using PortfolioMAD Object	6-74
Setting Average Turnover Constraints Using the PortfolioMAD Function	6-74
Setting Average Turnover Constraints Using the setTurnover Function	6-74
Working with One-Way Turnover Constraints Using PortfolioMAD Object	6-77
Setting One-Way Turnover Constraints Using the PortfolioMAD Function	6-77
Setting Turnover Constraints Using the setOneWayTurnover Function	6-77
Validate the MAD Portfolio Problem	6-80
Validating a MAD Portfolio Set	6-80
Validating MAD Portfolios	6-81
Estimate Efficient Portfolios Along the Entire Frontier for PortfolioMAD Object	6-84
Obtaining Portfolios Along the Entire Efficient Frontier	6-85
Obtaining Endpoints of the Efficient Frontier	6-87
Mixed-Integer MAD Portfolio Optimization Problem	6-89
Obtaining Efficient Portfolios for Target Returns	6-93
Obtaining Efficient Portfolios for Target Risks	6-96
Choosing and Controlling the Solver for PortfolioMAD Optimizations	6-99
Using 'TrustRegionCP' and 'ExtendedCP' SolverTypes	6-99
Using 'fmincon' SolverType	6-100
Using the Mixed Integer Nonlinear Programming (MINLP) Solver	6-101
Solver Guidelines for PortfolioMAD Objects	6-101
Estimate Efficient Frontiers for PortfolioMAD Object	6-105
Obtaining MAD Portfolio Risks and Returns	6-105
Obtaining the PortfolioMAD Standard Deviation	6-106
Plotting the Efficient Frontier for a PortfolioMAD Object	6-108
Postprocessing Results to Set Up Tradable Portfolios	6-113
Working with Other Portfolio Objects	6-115
Troubleshooting MAD Portfolio Optimization Results	6-118
PortfolioMAD Object Destroyed When Modifying	6-118
Matrix Incompatibility and "Non-Conformable" Errors	6-118
Missing Data Estimation Fails	6-118
mad_optim_transform Errors	6-118
Efficient Portfolios Do Not Make Sense	6-119

Performance Metrics Overview	7-2
Performance Metrics Types	7-2
Performance Metrics Illustration	7-3
Using the Sharpe Ratio	7-5
Using the Information Ratio	7-7
Using Tracking Error	7-8
Using Risk-Adjusted Return	7-9
Using Sample and Expected Lower Partial Moments	7-11
Introduction	7-11
Sample Lower Partial Moments	7-11
Expected Lower Partial Moments	7-12
Using Maximum and Expected Maximum Drawdown	7-14
Introduction	7-14
Maximum Drawdown	7-14
Expected Maximum Drawdown	7-16

Estimation of Transition Probabilities	8-2
Introduction	8-2
Estimate Transition Probabilities	8-2
Estimate Transition Probabilities for Different Rating Scales	8-4
Working with a Transition Matrix Containing NR Rating	8-6
Estimate Point-in-Time and Through-the-Cycle Probabilities	8-10
Estimate t-Year Default Probabilities	8-12
Estimate Bootstrap Confidence Intervals	8-13
Group Credit Ratings	8-15
Work with Nonsquare Matrices	8-17
Remove Outliers	8-18
Estimate Probabilities for Different Segments	8-19
Work with Large Datasets	8-20
Forecasting Corporate Default Rates	8-23
Credit Quality Thresholds	8-45
Introduction	8-45
Compute Credit Quality Thresholds	8-45
Visualize Credit Quality Thresholds	8-46

About Credit Scorecards	8-49
What Is a Credit Scorecard?	8-49
Credit Scorecard Development Process	8-50
Credit Scorecard Modeling Workflow	8-52
Credit Scorecard Modeling Using Observation Weights	8-55
Credit Scorecard Modeling with Missing Values	8-57
Troubleshooting Credit Scorecard Results	8-64
Predictor Name Is Unspecified and the Parser Returns an Error	8-64
Using bininfo or plotbins Before Binning	8-64
If Categorical Data Is Given as Numeric	8-66
NaNs Returned When Scoring a “Test” Dataset	8-68
Case Study for Credit Scorecard Analysis	8-71
Credit Scorecards with Constrained Logistic Regression Coefficients ..	8-91
Credit Default Swap (CDS)	8-100
Bootstrapping a Default Probability Curve	8-101
Finding Breakeven Spread for New CDS Contract	8-104
Valuing an Existing CDS Contract	8-107
Converting from Running to Upfront	8-109
Bootstrapping from Inverted Market Curves	8-111
Visualize Transitions Data for transprob	8-114
Impute Missing Data in the Credit Scorecard Workflow Using the k-Nearest Neighbors Algorithm	8-121
Impute Missing Data in the Credit Scorecard Workflow Using the Random Forest Algorithm	8-127
Treat Missing Data in a Credit Scorecard Workflow Using MATLAB fillmissing	8-132

Regression with Missing Data

9

Multivariate Normal Regression	9-2
Introduction	9-2
Multivariate Normal Linear Regression	9-2
Maximum Likelihood Estimation	9-3
Special Case of Multiple Linear Regression Model	9-4

Least-Squares Regression	9-4
Mean and Covariance Estimation	9-4
Convergence	9-4
Fisher Information	9-4
Statistical Tests	9-5
Maximum Likelihood Estimation with Missing Data	9-7
Introduction	9-7
ECM Algorithm	9-7
Standard Errors	9-8
Data Augmentation	9-8
Multivariate Normal Regression Functions	9-10
Multivariate Normal Regression Without Missing Data	9-11
Multivariate Normal Regression With Missing Data	9-11
Least-Squares Regression With Missing Data	9-11
Multivariate Normal Parameter Estimation With Missing Data	9-12
Support Functions	9-12
Multivariate Normal Regression Types	9-13
Regressions	9-13
Multivariate Normal Regression	9-13
Multivariate Normal Regression Without Missing Data	9-13
Multivariate Normal Regression With Missing Data	9-14
Least-Squares Regression	9-14
Least-Squares Regression Without Missing Data	9-14
Least-Squares Regression With Missing Data	9-14
Covariance-Weighted Least Squares	9-14
Covariance-Weighted Least Squares Without Missing Data	9-15
Covariance-Weighted Least Squares With Missing Data	9-15
Feasible Generalized Least Squares	9-15
Feasible Generalized Least Squares Without Missing Data	9-15
Feasible Generalized Least Squares With Missing Data	9-16
Seemingly Unrelated Regression	9-16
Seemingly Unrelated Regression Without Missing Data	9-17
Seemingly Unrelated Regression With Missing Data	9-17
Mean and Covariance Parameter Estimation	9-17
Troubleshooting Multivariate Normal Regression	9-18
Biased Estimates	9-18
Requirements	9-18
Slow Convergence	9-18
Nonrandom Residuals	9-19
Nonconvergence	9-19
Portfolios with Missing Data	9-21
Valuation with Missing Data	9-26
Introduction	9-26
Capital Asset Pricing Model	9-26
Estimation of the CAPM	9-27
Estimation with Missing Data	9-27
Estimation of Some Technology Stock Betas	9-27
Grouped Estimation of Some Technology Stock Betas	9-30
References	9-32

Capital Asset Pricing Model with Missing Data	9-34
---	------

Solving Sample Problems

10

Sensitivity of Bond Prices to Interest Rates	10-2
Bond Portfolio for Hedging Duration and Convexity	10-6
Bond Prices and Yield Curve Parallel Shifts	10-9
Bond Prices and Yield Curve Nonparallel Shifts	10-12
Greek-Neutral Portfolios of European Stock Options	10-14
Term Structure Analysis and Interest-Rate Swaps	10-18
Plotting an Efficient Frontier Using portopt	10-22
Plotting Sensitivities of an Option	10-25
Plotting Sensitivities of a Portfolio of Options	10-27
Bond Portfolio Optimization Using Portfolio Object	10-30
Hedge Options Using Reinforcement Learning Toolbox	10-40
Hedge Using Monte Carlo Simulation	10-49
Using Extreme Value Theory and Copula Fitting to Generate Synthetic Data	10-61

Using Financial Timetables

11

Convert Financial Time Series Objects (fints) to Timetables	11-2
Create Time Series	11-2
Index an Object	11-3
Transform Time Series	11-3
Convert Time Series	11-4
Merge Time Series	11-5
Analyze Time Series	11-5
Data Extraction	11-6
Use Timetables in Finance	11-7

12

Trading Calendars User Interface	12-2
UICalendar User Interface	12-4
Using UICalendar in Standalone Mode	12-4
Using UICalendar with an Application	12-4

Technical Analysis**13**

Technical Indicators	13-2
-----------------------------------	-------------

Stochastic Differential Equations**14**

SDEs	14-2
SDE Modeling	14-2
Trials vs. Paths	14-3
NTrials, NPeriods, and NSteps	14-3
SDE Class Hierarchy	14-5
SDE Models	14-7
Introduction	14-7
Creating SDE Objects	14-7
Drift and Diffusion	14-10
Available SDE Models	14-11
SDE Simulation and Interpolation Methods	14-13
Base SDE Models	14-16
Overview	14-16
Specify Base Stochastic Differential Equation (SDE) Model	14-16
Drift and Diffusion Models	14-19
Overview	14-19
Specify Drift and Diffusion Rate Functions	14-19
Specify SDEDDO with Drift and Diffusion Functions	14-20
Linear Drift Models	14-22
Overview	14-22
Specify SDELD Model	14-22
Parametric Models	14-24
Creating Brownian Motion (BM) Models	14-24
Specify Brownian Motion Model	14-24
Creating Constant Elasticity of Variance (CEV) Models	14-25

Creating Geometric Brownian Motion (GBM) Models	14-25
Creating Stochastic Differential Equations from Mean-Reverting Drift (SDEMRD) Models	14-26
Creating Cox-Ingersoll-Ross (CIR) Square Root Diffusion Models	14-27
Creating Hull-White/Vasicek (HWV) Gaussian Diffusion Models	14-28
Creating Heston Stochastic Volatility Models	14-29
Simulating Equity Prices	14-31
Simulating Multidimensional Market Models	14-31
Induce Dependence and Correlation Between States	14-41
Dynamic Behavior of Market Parameters	14-43
Price European Stock Options Using Monte Carlo Simulation	14-47
Simulating Interest Rates	14-50
Simulate Interest Rates Through Interpolation	14-50
Simulate Positive Interest Rates	14-54
Stratified Sampling	14-58
Quasi-Monte Carlo Simulation	14-63
Performance Considerations	14-65
Managing Memory	14-65
Enhancing Performance	14-66
Optimizing Accuracy: About Solution Precision and Error	14-66
Price American Basket Options Using Standard Monte Carlo and Quasi- Monte Carlo Simulation	14-71
Volatility Modeling for Soft Commodities	14-88
Improving Performance of Monte Carlo Simulation with Parallel Computing	14-111

Functions

15

Bibliography

A

Bibliography	A-2
Bond Pricing and Yields	A-2
Term Structure of Interest Rates	A-2
Derivatives Pricing and Yields	A-3
Portfolio Analysis	A-3
Investment Performance Metrics	A-3
Financial Statistics	A-4
Standard References	A-4
Credit Risk Analysis	A-5

Credit Derivatives	A-5
Portfolio Optimization	A-5
Stochastic Differential Equations	A-6
Life Tables	A-6

Getting Started

- “Financial Toolbox Product Description” on page 1-2
- “Expected Users” on page 1-3
- “Analyze Sets of Numbers Using Matrix Functions” on page 1-4
- “Matrix Algebra Refresher” on page 1-7
- “Using Input and Output Arguments with Functions” on page 1-15

Financial Toolbox Product Description

Analyze financial data and develop financial models

Financial Toolbox provides functions for the mathematical modeling and statistical analysis of financial data. You can analyze, backtest, and optimize investment portfolios taking into account turnover, transaction costs, semi-continuous constraints, and minimum or maximum number of assets. The toolbox enables you to estimate risk, model credit scorecards, analyze yield curves, price fixed-income instruments and European options, and measure investment performance.

Stochastic differential equation (SDE) tools let you model and simulate a variety of stochastic processes. Time series analysis functions let you perform transformations or regressions with missing data and convert between different trading calendars and day-count conventions.

Expected Users

In general, this guide assumes experience working with financial derivatives and some familiarity with the underlying models.

In designing Financial Toolbox documentation, we assume that your title is like one of these:

- Analyst, quantitative analyst
- Risk manager
- Portfolio manager
- Asset allocator
- Financial engineer
- Trader
- Student, professor, or other academic

We also assume that your background, education, training, and responsibilities match some aspects of this profile:

- Finance, economics, perhaps accounting
- Engineering, mathematics, physics, other quantitative sciences
- Focus on quantitative approaches to financial problems

Analyze Sets of Numbers Using Matrix Functions

In this section...

"Introduction" on page 1-4

"Key Definitions" on page 1-4

"Referencing Matrix Elements" on page 1-4

"Transposing Matrices" on page 1-5

Introduction

Many financial analysis procedures involve *sets* of numbers; for example, a portfolio of securities at various prices and yields. Matrices, matrix functions, and matrix algebra are the most efficient ways to analyze sets of numbers and their relationships. Spreadsheets focus on individual cells and the relationships between cells. While you can think of a set of spreadsheet cells (a range of rows and columns) as a matrix, a matrix-oriented tool like MATLAB® software manipulates sets of numbers more quickly, easily, and naturally. For more information, see "Matrix Algebra Refresher" on page 1-7.

Key Definitions

Matrix

A rectangular array of numeric or algebraic quantities subject to mathematical operations; the regular formation of elements into rows and columns. Described as a "*m*-by-*n*" matrix, with *m* the number of rows and *n* the number of columns. The description is always "row-by-column." For example, here is a 2-by-3 matrix of two bonds (the rows) with different par values, coupon rates, and coupon payment frequencies per year (the columns) entered using MATLAB notation:

```
Bonds = [1000    0.06    2
          500    0.055   4]
```

Vector

A matrix with only one row or column. Described as a "1-by-*n*" or "*m*-by-1" matrix. The description is always "row-by-column." For example, here is a 1-by-4 vector of cash flows in MATLAB notation:

```
Cash = [1500    4470    5280   -1299]
```

Scalar

A 1-by-1 matrix; that is, a single number.

Referencing Matrix Elements

To reference specific matrix elements, use (row, column) notation. For example:

```
Bonds(1,2)
```

```
ans =
```

```
0.06
```

```
Cash(3)
```

```
ans =
```

```
5280.00
```

You can enlarge matrices using small matrices or vectors as elements. For example,

```
AddBond = [1000 0.065 2];
Bonds = [Bonds; AddBond]
```

adds another row to the matrix and creates

```
Bonds =
```

```
1000 0.06 2
500 0.055 4
1000 0.065 2
```

Likewise,

```
Prices = [987.50
475.00
995.00]
```

```
Bonds = [Prices, Bonds]
```

adds another column and creates

```
Bonds =
```

```
987.50 1000 0.06 2
475.00 500 0.055 4
995.00 1000 0.065 2
```

Finally, the colon (:) is important in generating and referencing matrix elements. For example, to reference the par value, coupon rate, and coupon frequency of the second bond:

```
BondItems = Bonds(2, 2:4)
```

```
BondItems =
```

```
500.00 0.055 4
```

Transposing Matrices

Sometimes matrices are in the wrong configuration for an operation. In MATLAB, the apostrophe or prime character (') transposes a matrix: columns become rows, rows become columns. For example,

```
Cash = [1500 4470 5280 -1299]'
```

produces

```
Cash =
```

```
1500
4470
5280
-1299
```

See Also

More About

- “Matrix Algebra Refresher” on page 1-7
- “Using Input and Output Arguments with Functions” on page 1-15

Matrix Algebra Refresher

In this section...

"Introduction" on page 1-7
 "Adding and Subtracting Matrices" on page 1-7
 "Multiplying Matrices" on page 1-8
 "Dividing Matrices" on page 1-11
 "Solving Simultaneous Linear Equations" on page 1-11
 "Operating Element by Element" on page 1-13

Introduction

The explanations in the sections that follow should help refresh your skills for using matrix algebra and using MATLAB functions.

In addition, *Macro-Investment Analysis* by William Sharpe also provides an excellent explanation of matrix algebra operations using MATLAB. It is available on the web at:

<https://www.stanford.edu/~wfsharpe/mia/mia.htm>

Tip When you are setting up a problem, it helps to "talk through" the units and dimensions associated with each input and output matrix. In the example under "Multiplying Matrices" on page 1-8, one input matrix has five days' closing prices for three stocks, the other input matrix has shares of three stocks in two portfolios, and the output matrix therefore has five days' closing values for two portfolios. It also helps to name variables using descriptive terms.

Adding and Subtracting Matrices

Matrix addition and subtraction operate element-by-element. The two input matrices must have the same dimensions. The result is a new matrix of the same dimensions where each element is the sum or difference of each corresponding input element. For example, consider combining portfolios of different quantities of the same stocks ("shares of stocks A, B, and C [the rows] in portfolios P and Q [the columns] plus shares of A, B, and C in portfolios R and S").

```
Portfolios_PQ = [100    200
                 500    400
                 300    150];
```

```
Portfolios_RS = [175    125
                 200    200
                 100    500];
```

```
NewPortfolios = Portfolios_PQ + Portfolios_RS
```

```
NewPortfolios =
```

```
    275    325
    700    600
    400    650
```

Adding or subtracting a scalar and a matrix is allowed and also operates element-by-element.

```
SmallerPortf = NewPortfolios-10
```

```
SmallerPortf =  
    265.00    315.00  
    690.00    590.00  
    390.00    640.00
```

Multiplying Matrices

Matrix multiplication does *not* operate element-by-element. It operates according to the rules of linear algebra. In multiplying matrices, it helps to remember this key rule: the inner dimensions must be the same. That is, if the first matrix is m -by- 3 , the second must be 3 -by- n . The resulting matrix is m -by- n . It also helps to “talk through” the units of each matrix, as mentioned in “Analyze Sets of Numbers Using Matrix Functions” on page 1-4.

Matrix multiplication also is *not* commutative; that is, it is not independent of order. $A*B$ does *not* equal $B*A$. The dimension rule illustrates this property. If A is 1 -by- 3 matrix and B is 3 -by- 1 matrix, $A*B$ yields a scalar (1 -by- 1) matrix but $B*A$ yields a 3 -by- 3 matrix.

Multiplying Vectors

Vector multiplication follows the same rules and helps illustrate the principles. For example, a stock portfolio has three different stocks and their closing prices today are:

```
ClosePrices = [42.5    15    78.875]
```

The portfolio contains these numbers of shares of each stock.

```
NumShares = [100  
             500  
             300]
```

To find the value of the portfolio, multiply the vectors

```
PortfValue = ClosePrices * NumShares
```

which yields:

```
PortfValue =  
  
    3.5413e+004
```

The vectors are 1 -by- 3 and 3 -by- 1 ; the resulting vector is 1 -by- 1 , a scalar. Multiplying these vectors thus means multiplying each closing price by its respective number of shares and summing the result.

To illustrate order dependence, switch the order of the vectors

```
Values = NumShares * ClosePrices
```

```
Values =  
  
    1.0e+004 *  
  
    0.4250    0.1500    0.7887  
    2.1250    0.7500    3.9438  
    1.2750    0.4500    2.3663
```

which shows the closing values of 100, 500, and 300 shares of each stock, not the portfolio value, and this is meaningless for this example.

Computing Dot Products of Vectors

In matrix algebra, if X and Y are vectors of the same length

$$Y = [y_1, y_2, \dots, y_n]$$

$$X = [x_1, x_2, \dots, x_n]$$

then the dot product

$$X \cdot Y = x_1y_1 + x_2y_2 + \dots + x_ny_n$$

is the scalar product of the two vectors. It is an exception to the commutative rule. To compute the dot product in MATLAB, use `sum(X .* Y)` or `sum(Y .* X)`. Be sure that the two vectors have the same dimensions. To illustrate, use the previous vectors.

```
Value = sum(NumShares .* ClosePrices')
```

```
Value =
```

```
3.5413e+004
```

```
Value = sum(ClosePrices .* NumShares')
```

```
Value =
```

```
3.5413e+004
```

As expected, the value in these cases matches the `PortfValue` computed previously.

Multiplying Vectors and Matrices

Multiplying vectors and matrices follows the matrix multiplication rules and process. For example, a portfolio matrix contains closing prices for a week. A second matrix (vector) contains the stock quantities in the portfolio.

```
WeekClosePr = [42.5      15      78.875
               42.125   15.5     78.75
               42.125   15.125   79
               42.625   15.25    78.875
               43       15.25    78.625];
PortQuan = [100
            500
            300];
```

To see the closing portfolio value for each day, simply multiply

```
WeekPortValue = WeekClosePr * PortQuan
```

```
WeekPortValue =
```

```
1.0e+004 *
```

```
3.5412
```

```
3.5587
```

```

3.5475
3.5550
3.5513

```

The prices matrix is 5-by-3, the quantity matrix (vector) is 3-by-1, so the resulting matrix (vector) is 5-by-1.

Multiplying Two Matrices

Matrix multiplication also follows the rules of matrix algebra. In matrix algebra notation, if A is an m -by- n matrix and B is an n -by- p matrix

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ \vdots & \vdots & & \vdots \\ a_{i1} & a_{i2} & \cdots & a_{in} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}, \quad B = \begin{bmatrix} b_{11} & \cdots & b_{1j} & \cdots & b_{1p} \\ b_{21} & \cdots & b_{2j} & \cdots & b_{2p} \\ \vdots & & \vdots & & \vdots \\ b_{n1} & \cdots & b_{nj} & \cdots & b_{np} \end{bmatrix}$$

then $C = A*B$ is an m -by- p matrix; and the element c_{ij} in the i th row and j th column of C is

$$c_{ij} = a_{i1}b_{1j} + a_{i2}b_{2j} + \cdots + a_{in}b_{nj}.$$

To illustrate, assume that there are two portfolios of the same three stocks previously mentioned but with different quantities.

```

Portfolios = [100    200
               500    400
               300    150];

```

Multiplying the 5-by-3 week's closing prices matrix by the 3-by-2 portfolios matrix yields a 5-by-2 matrix showing each day's closing value for both portfolios.

```
PortfolioValues = WeekClosePr * Portfolios
```

```
PortfolioValues =
```

```
1.0e+004 *
```

```

3.5412    2.6331
3.5587    2.6437
3.5475    2.6325
3.5550    2.6456
3.5513    2.6494

```

Monday's values result from multiplying each Monday closing price by its respective number of shares and summing the result for the first portfolio, then doing the same for the second portfolio. Tuesday's values result from multiplying each Tuesday closing price by its respective number of shares and summing the result for the first portfolio, then doing the same for the second portfolio. And so on, through the rest of the week. With one simple command, MATLAB quickly performs many calculations.

Multiplying a Matrix by a Scalar

Multiplying a matrix by a scalar is an exception to the dimension and commutative rules. It just operates element-by-element.

```
Portfolios = [100  200
              500  400
              300  150];
```

```
DoublePort = Portfolios * 2
```

```
DoublePort =
    200    400
   1000    800
    600    300
```

Dividing Matrices

Matrix division is useful primarily for solving equations, and especially for solving simultaneous linear equations (see “Solving Simultaneous Linear Equations” on page 1-11). For example, you want to solve for X in $A*X = B$.

In ordinary algebra, you would divide both sides of the equation by A , and X would equal B/A . However, since matrix algebra is not commutative ($A*X \neq X*A$), different processes apply. In formal matrix algebra, the solution involves matrix inversion. MATLAB, however, simplifies the process by providing two matrix division symbols, left and right ($\backslash$ and $/$). In general,

$X = A \backslash B$ solves for X in $A*X = B$ and

$X = B/A$ solves for X in $X*A = B$.

In general, matrix A must be a nonsingular square matrix; that is, it must be invertible and it must have the same number of rows and columns. (Generally, a matrix is invertible if the matrix times its inverse equals the identity matrix. To understand the theory and proofs, consult a textbook on linear algebra such as *Elementary Linear Algebra* by Hill listed in “Bibliography” on page A-2.) MATLAB gives a warning message if the matrix is singular or nearly so.

Solving Simultaneous Linear Equations

Matrix division is especially useful in solving simultaneous linear equations. Consider this problem: Given two portfolios of mortgage-based instruments, each with certain yields depending on the prime rate, how do you weight the portfolios to achieve certain annual cash flows? The answer involves solving two linear equations.

A linear equation is any equation of the form

$$a_1x + a_2y = b,$$

where a_1 , a_2 , and b are constants (with a_1 and a_2 not both 0), and x and y are variables. (It is a linear equation because it describes a line in the xy -plane. For example, the equation $2x + y = 8$ describes a line such that if $x = 2$, then $y = 4$.)

A system of linear equations is a set of linear equations that you usually want to solve at the same time; that is, simultaneously. A basic principle for exact answers in solving simultaneous linear equations requires that there be as many equations as there are unknowns. To get exact answers for x and y , there must be two equations. For example, to solve for x and y in the system of linear equations

$$\begin{aligned} 2x + y &= 13 \\ x - 3y &= -18, \end{aligned}$$

there must be two equations, which there are. Matrix algebra represents this system as an equation involving three matrices: A for the left-side constants, X for the variables, and B for the right-side constants

$$A = \begin{bmatrix} 2 & 1 \\ 1 & -3 \end{bmatrix}, \quad X = \begin{bmatrix} x \\ y \end{bmatrix}, \quad B = \begin{bmatrix} 13 \\ -18 \end{bmatrix},$$

where $A \cdot X = B$.

Solving the system simultaneously means solving for X . Using MATLAB,

$$A = \begin{bmatrix} 2 & 1 \\ 1 & -3 \end{bmatrix};$$

$$B = \begin{bmatrix} 13 \\ -18 \end{bmatrix};$$

$$X = A \setminus B$$

solves for X in $A \cdot X = B$.

$$X = \begin{bmatrix} 3 & 7 \end{bmatrix}$$

So $x = 3$ and $y = 7$ in this example. In general, you can use matrix algebra to solve any system of linear equations such as

$$a_{11}x_1 + a_{12}x_2 + \dots + a_{1n}x_n = b_1$$

$$a_{21}x_1 + a_{22}x_2 + \dots + a_{2n}x_n = b_2$$

$$\vdots$$

$$a_{m1}x_1 + a_{m2}x_2 + \dots + a_{mn}x_n = b_m$$

by representing them as matrices

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix}, \quad X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}, \quad B = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix}$$

and solving for X in $A \cdot X = B$.

To illustrate, consider this situation. There are two portfolios of mortgage-based instruments, M1 and M2. They have current annual cash payments of \$100 and \$70 per unit, respectively, based on today's prime rate. If the prime rate moves down one percentage point, their payments would be \$80 and \$40. An investor holds 10 units of M1 and 20 units of M2. The investor's receipts equal cash payments times units, or $R = C \cdot U$, for each prime-rate scenario. As word equations:

	M1	M2
Prime flat:	\$100 * 10 units	+ \$70 * 20 units = \$2400 receipts
Prime down:	\$80 * 10 units	+ \$40 * 20 units = \$1600 receipts

As MATLAB matrices:

```

Cash = [100  70
        80  40];

Units = [10
        20];

Receipts = Cash * Units

Receipts =

    2400
    1600

```

Now the investor asks this question: Given these two portfolios and their characteristics, how many units of each should they hold to receive \$7000 if the prime rate stays flat and \$5000 if the prime drops one percentage point? Find the answer by solving two linear equations.

	M1	M2
Prime flat:	\$100 * x units	+ \$70 * y units = \$7000 receipts
Prime down:	\$80 * x units	+ \$40 * y units = \$5000 receipts

In other words, solve for U (units) in the equation $R(\text{receipts}) = C(\text{cash}) * U(\text{units})$. Using MATLAB left division

```

Cash = [100  70
        80  40];

Receipts = [7000
           5000];

Units = Cash \ Receipts

Units =

    43.7500
    37.5000

```

The investor should hold 43.75 units of portfolio M1 and 37.5 units of portfolio M2 to achieve the annual receipts desired.

Operating Element by Element

Finally, element-by-element arithmetic operations are called operations. To indicate a MATLAB array operation, precede the operator with a period (.). Addition and subtraction, and matrix multiplication and division by a scalar, are already array operations so no period is necessary. When using array operations on two matrices, the dimensions of the matrices must be the same. For example, given vectors of stock dividends and closing prices

```

Dividends = [1.90  0.40  1.56  4.50];
Prices = [25.625  17.75  26.125  60.50];

Yields = Dividends ./ Prices

```

Yields =

0.0741 0.0225 0.0597 0.0744

See Also

More About

- “Analyze Sets of Numbers Using Matrix Functions” on page 1-4
- “Using Input and Output Arguments with Functions” on page 1-15

Using Input and Output Arguments with Functions

In this section...

"Input Arguments" on page 1-15

"Output Arguments" on page 1-16

Input Arguments

Vector and Matrix Input

By design, MATLAB software can efficiently perform repeated operations on collections of data stored in vectors and matrices. MATLAB code that is written to operate simultaneously on different arrays is said to be vectorized. Vectorized code is not only clean and concise, but is also efficiently processed by MATLAB.

Because MATLAB is optimized for processing vectorized code, many Financial Toolbox functions accept either vector or matrix input arguments, rather than single (scalar) values.

One example of such a function is the `irr` function, which computes the internal rate of return of a cash flow stream. If you input a vector of cash flows from a single cash flow stream, then `irr` returns a scalar rate of return. If you input a matrix of cash flows from multiple cash flow streams, where each matrix column represents a different stream, then `irr` returns a vector of internal rates of return, where the columns correspond to the columns of the input matrix. Many other Financial Toolbox functions work similarly.

As an example, suppose that you make an initial investment of \$100, from which you then receive by a series of annual cash receipts of \$10, \$20, \$30, \$40, and \$50. This cash flow stream is stored in a vector

```
CashFlows = [-100 10 20 30 40 50]'
```

```
CashFlows =
-100
  10
  20
  30
  40
  50
```

Use the `irr` function to compute the internal rate of return of the cash flow stream.

```
Rate = irr(CashFlows)
```

```
Rate =
0.1201
```

For the single cash flow stream `CashFlows`, the function returns a scalar rate of return of 0.1201, or 12.01%.

Now, use the `irr` function to compute internal rates of return for multiple cash flow streams.

```
Rate = irr([CashFlows CashFlows CashFlows])
```

```
Rate =  
    0.1201    0.1201    0.1201
```

MATLAB performs the same computation on all the assets at once. For the three cash flow streams, the `irr` function returns a vector of three internal rates of return.

In the Financial Toolbox context, vectorized programming is useful in portfolio management. You can organize multiple assets into a single collection by placing data for each asset in a different matrix column or row, then pass the matrix to a Financial Toolbox function.

Character Vector Input

Enter MATLAB character vectors surrounded by single quotes ('character vector').

A character vector is stored as a character array, one ASCII character per element. Thus, the date character vector is

```
DateCharacterVector = '9/16/2017'
```

This date character vector is actually a 1-by-9 vector. If you create a vector or matrix of character vectors, each character vector must have the same length. Using a column vector to create a vector of character vectors can allow you to visually check that all character vectors are the same length. If your character vectors are not the same length, use spaces or zeros to make them the same length, as in the following code.

```
DateFields = ['01/12/2017'  
              '02/14/2017'  
              '03/03/2017'  
              '06/14/2017'  
              '12/01/2017'];
```

`DateFields` is a 5-by-10 array of character vectors.

You cannot mix numbers and character vectors in a vector or matrix. If you input a vector or matrix that contains a mix of numbers and character vectors, MATLAB treats every entry as a character. As an example, input the following code

```
Item = [83  90  99  '14-Sep-1999']
```

```
Item =
```

```
SZc14-Sep-1999
```

The software understands the input not as a 1-by-4 vector, but as a 1-by-14 character array with the value `SZc14-Sep-1999`.

Output Arguments

Some functions return no arguments, some return just one, and some return multiple arguments. Functions that return multiple arguments use the syntax

```
[A, B, C] = function(input_arguments...)
```

to return arguments A, B, and C. If you omit all but one, the function returns the first argument. Thus, for this example if you use the syntax

```
X = function(input_arguments...)
```

the function returns a value for A, but not for B or C.

Some functions that return vectors accept only scalars as arguments. Such functions cannot accept vectors as arguments and return matrices, where each column in the output matrix corresponds to an entry in the input. Output vectors can be variable length.

For example, most functions that require asset life as an input, and return values corresponding to different periods over the asset life, cannot handle vectors or matrices as input arguments. These functions include `amortize`, `depfixdb`, `depgendb`, and `depsoyd`. For example, consider a car for which you want to compute the depreciation schedule. Use the `depfixdb` function to compute a stream of declining-balance depreciation values for the asset. Set the initial value of the asset and the lifetime of the asset. Note that in the returned vector, the asset lifetime determines the number of rows. Now consider a collection of cars with different lifetimes. Because `depfixdb` cannot output a matrix with an unequal number of rows in each column, `depfixdb` cannot accept a single input vector with values for each asset in the collection.

See Also

Related Examples

- “Matrices and Arrays”

More About

- “Analyze Sets of Numbers Using Matrix Functions” on page 1-4
- “Matrix Algebra Refresher” on page 1-7

Performing Common Financial Tasks

- “Handle and Convert Dates” on page 2-2
- “Analyzing and Computing Cash Flows” on page 2-11
- “Pricing and Computing Yields for Fixed-Income Securities” on page 2-15
- “Treasury Bills Defined” on page 2-25
- “Computing Treasury Bill Price and Yield” on page 2-26
- “Term Structure of Interest Rates” on page 2-29
- “Returns with Negative Prices” on page 2-32
- “Pricing and Analyzing Equity Derivatives” on page 2-39
- “About Life Tables” on page 2-44
- “Case Study for Life Tables Analysis” on page 2-46
- “Machine Learning for Statistical Arbitrage: Introduction” on page 2-48
- “Machine Learning for Statistical Arbitrage I: Data Management and Visualization” on page 2-50
- “Machine Learning for Statistical Arbitrage II: Feature Engineering and Model Development” on page 2-59
- “Machine Learning for Statistical Arbitrage III: Training, Tuning, and Prediction” on page 2-69
- “Backtest Deep Learning Model for Algorithmic Trading of Limit Order Book Data” on page 2-78

Handle and Convert Dates

In this section...
"Date Formats" on page 2-2
"Date Conversions" on page 2-3
"Current Date and Time" on page 2-7
"Determining Specific Dates" on page 2-8
"Determining Holidays" on page 2-8
"Determining Cash-Flow Dates" on page 2-9

Date Formats

Virtually all financial data derives from a time series, functions in Financial Toolbox have extensive date-handling capabilities. The toolbox functions support date or date-and-time formats as character vectors, datetime arrays, or serial date numbers.

- Date character vectors are text that represent date and time, which you can use with multiple formats. For example, 'dd-mmm-yyyy HH:MM:SS', 'dd-mmm-yyyy', and 'mm/dd/yyyy' are all supported text formats for a date character vector. Most often, you work with date character vectors (such as 14-Sep-1999) when dealing with dates.
- Datetime arrays, created using `datetime`, are the best data type for representing points in time. `datetime` values have flexible display formats and up to nanosecond precision, and can account for time zones, daylight saving time, and leap seconds. When `datetime` objects are used as inputs to other Financial Toolbox functions, the format of the input `datetime` object is preserved. For example:

```
originalDate = datetime('now','Format','yyyy-MM-dd HH:mm:ss');  
% Find the next business day  
b = busdate(originalDate)
```

```
b =
```

```
datetime
```

```
2021-05-04 15:59:34
```

- Serial date numbers represent a calendar date as the number of days that have passed since a fixed base date. In MATLAB software, serial date number 1 is January 1,0000 A.D. Financial Toolbox works internally with serial date numbers (such as, 730377). MATLAB also uses serial time to represent fractions of days beginning at midnight. For example, 6 p.m. equals 0.75 serial days, so 6:00 p.m. on 14-Sep-1999, in MATLAB, is serial date number 730377.75

Note If you specify a two-digit year, MATLAB assumes that the year lies within the 100-year period centered on the current year. See the function `datenum` for specific information. MATLAB internal date handling and calculations generate no ambiguous values. However, whenever possible, use serial date numbers or date character vectors containing four-digit years.

Many Financial Toolbox functions that require dates as input arguments accept date character vectors, datetime arrays, or serial date numbers. If you are dealing with a few dates at the MATLAB command-line level, date character vectors are more convenient. If you are using Financial Toolbox functions on large numbers of dates, as in analyzing large portfolios or cash flows, performance improves if you use datetime arrays or serial date numbers. For more information, see “Represent Dates and Times in MATLAB”.

Date Conversions

Financial Toolbox provides functions that convert date character vectors to or from serial date numbers. In addition, you can convert character vectors or serial date numbers to datetime arrays.

Functions that convert between date formats are:

<code>datedisp</code>	Displays a numeric matrix with date entries formatted as date character vectors.
<code>datenum</code>	Converts a date character vector to a serial date number.
<code>datestr</code>	Converts a serial date number to a date character vector.
<code>datetime</code>	Converts from date character vectors or serial date numbers to create a datetime array.
<code>datevec</code>	Converts a serial date number or date character vector to a date vector whose elements are [Year Month Day Hour Minute Second].
<code>m2xdate</code>	Converts MATLAB serial date number to Excel® serial date number.
<code>x2mdate</code>	Converts Microsoft® Excel serial date number to MATLAB serial date number.

For more information, see “Convert Between Text and datetime or duration Values”.

Convert Between Datetime Arrays and Character Vectors

A date can be a character vector composed of fields related to a specific date and time. There are several ways to represent dates and times in several text formats. For example, all the following are character vectors represent August 23, 2010 at 04:35:42 PM:

```
'23-Aug-2010 04:35:06 PM'
'Wednesday, August 23'
'08/23/10 16:35'
'Aug 23 16:35:42.946'
```

A date character vector includes characters that separate the fields, such as the hyphen, space, and colon used here:

```
d = '23-Aug-2010 16:35:42'
```

Convert one or more date character vectors to a `datetime` array using the `datetime` function. For the best performance, specify the format of the input character vectors as an input to `datetime`.

Note The specifiers that `datetime` uses to describe date and time formats differ from the specifiers that the `datestr`, `datevec`, and `datenum` functions accept.

```
t = datetime(d, 'InputFormat', 'dd-MMM-yyyy HH:mm:ss')
t =
    23-Aug-2010 16:35:42
```

Although the date string, `d`, and the `datetime` scalar, `t`, look similar, they are not equal. View the size and data type of each variable.

```
whos d t
```

Name	Size	Bytes	Class	Attributes
d	1x20	40	char	
t	1x1	121	datetime	

Convert a `datetime` array to a character vector that uses `char` or `cellstr`. For example, convert the current date and time to a timestamp to append to a file name.

```
t = datetime('now', 'Format', 'yyyy-MM-dd' 'T' 'HHmmss')
t =
    datetime
    2016-12-11T125628

S = char(t);
filename = ['myTest_', S]
filename =
    'myTest_2016-12-11T125628'
```

Convert Serial Date Numbers to Datetime Arrays

Serial time can represent fractions of days beginning at midnight. For example, 6 p.m. equals 0.75 serial days, so the character vector '31-Oct-2003, 6:00 PM' in MATLAB is date number 731885.75.

Convert one or more serial date numbers to a `datetime` array using the `datetime` function. Specify the type of date number that is being converted:

```
t = datetime(731885.75, 'ConvertFrom', 'datenum')
t =
    datetime
    31-Oct-2003 18:00:00
```

Convert Datetime Arrays to Numeric Values

Some MATLAB functions accept numeric data types but not `datetime` values as inputs. To apply these functions to your date and time data, first, convert `datetime` values to meaningful numeric values, and then call the function. For example, the `log` function accepts `double` inputs but not `datetime` inputs. Suppose that you have a `datetime` array of dates spanning the course of a research study or experiment.


```
t = datetime(2014,6,18) + calmonths(1:4)
t =
    1×4 datetime array

    18-Jul-2014    18-Aug-2014    18-Sep-2014    18-Oct-2014
```

Subtract the origin value. For example, the origin value can be the starting day of an experiment.

```
dt = t - datetime(2014,7,1)
dt =
    1×4 duration array

    408:00:00    1152:00:00    1896:00:00    2616:00:00
```

`dt` is a `duration` array. Convert `dt` to a double array of values in units of years, days, hours, minutes, or seconds by using the `years`, `days`, `hours`, `minutes`, or `seconds` function, respectively.

```
x = hours(dt)
x =
    408    1152    1896    2616
```

Pass the double array as the input to the `log` function.

```
y = log(x)
y =
    6.0113    7.0493    7.5475    7.8694
```

Input Conversions with `datenum`

The `datenum` function is important for using Financial Toolbox software efficiently. `datenum` takes an input date character vector in any of several formats, with `'dd-mmm-yyyy'`, `'mm/dd/yyyy'`, or `'dd-mmm-yyyy, hh:mm:ss.ss'` formats being the most common. The input date character vector can have up to six fields formed by letters and numbers separated by any other characters, such that:

- The day field is an integer from 1 through 31.
- The month field is either an integer from 1 through 12 or an alphabetical character vector with at least three characters.
- The year field is a nonnegative integer. If only two numbers are specified, then the year is assumed to lie within the 100-year period centered on the current year. If the year is omitted, the current year is the default.
- The hours, minutes, and seconds fields are optional. They are integers separated by colons or followed by `'am'` or `'pm'`.

For example, if the current year is 1999, then all these dates are equivalent:

```
'17-May-1999'
'17-May-99'
'17-may'
'May 17, 1999'
```

```
'5/17/99'  
'5/17'
```

Also, both of these formats represent the same time.

```
'17-May-1999, 18:30'  
'5/17/99/6:30 pm'
```

The default format for numbers-only input follows the US convention. Therefore, 3/6 is March 6, not June 3.

With `datenum`, you can convert dates into serial date format, store them in a matrix variable, and then later pass the variable to a function. Alternatively, you can use `datenum` directly in a function input argument list.

For example, consider the function `bndprice` that computes the price of a bond given the yield to maturity. First set up variables for the yield to maturity, coupon rate, and the necessary dates.

```
Yield      = 0.07;  
CouponRate = 0.08;  
Settle     = datenum('17-May-2000');  
Maturity   = datenum('01-Oct-2000');
```

Then call the function with the variables.

```
bndprice(Yield,CouponRate,Settle,Maturity)
```

```
ans =  
  
100.3503
```

Alternatively, convert date character vectors to serial date numbers directly in the function input argument list.

```
bndprice(0.07,0.08,datenum('17-May-2000'),...  
datenum('01-Oct-2000'))
```

```
ans =  
  
100.3503
```

`bndprice` is an example of a function designed to detect the presence of date character vectors and make the conversion automatically. For functions like `bndprice`, date character vectors can be passed directly.

```
bndprice(0.07,0.08,'17-May-2000','01-Oct-2000')
```

```
ans =  
  
100.3503
```

The decision to represent dates as either date character vectors or serial date numbers is often a matter of convenience. For example, when formatting data for visual display or for debugging date-handling code, you can view dates more easily as date character vectors because serial date numbers are difficult to interpret. Alternately, serial date numbers are just another type of numeric data, which you can place in a matrix along with any other numeric data for convenient manipulation.

Remember that if you create a vector of input date character vectors, use a column vector, and be sure that all character vectors are the same length. To ensure that the character vectors are the same